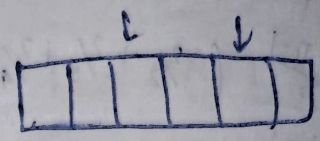
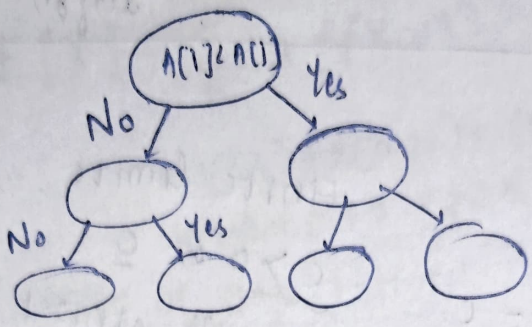
18/2/22

Sorting

Any comparison sort takes $\Omega(n \log n)$ time in the worst case

Comparison sort





$T(n)$

No. of comparisons the algorithm makes

No. of leaves $\leq 2^{T(n)}$

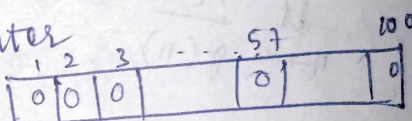
$z_1 < z_2 < \dots z_n$

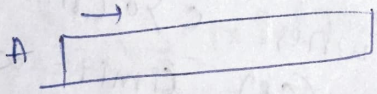
$$2^{T(n)} \geq n! > (n/e)^n$$

$$T(n) > n \log_2 (n/e) = n \log_2 n - n \log_2 e$$

$$= \Omega(n \log n)$$

Counting sort:





Initiale c[1..100] with all zeroes

Input array → A

Output array - B

```
for (i=0 ; i<n ; i++)
    c[A[i]] ++ ;
```

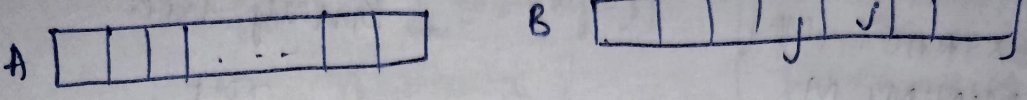
c[i] → no. of times i occured in A

```
i ← 0
for( j=1 ; j< K ; j++)
    for (i=0, i < c[j] ; i++) {
        Add j to B
```

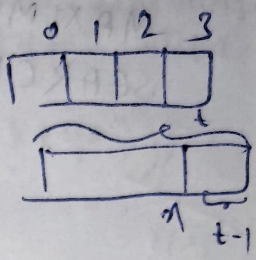
```
for (i=2 ; i ≤ K ; i++) {
    c[i] ← c[i] + c[i-1]
```

Modify C so that C[i] is the no. of elements in A whose value $\leq i$.

C[i] C[j]--;




```
for(i=n-1; i>=0; i--){
    B[C[A[i]]-1] ← A[i]
    C[A[i]]--;
}
```



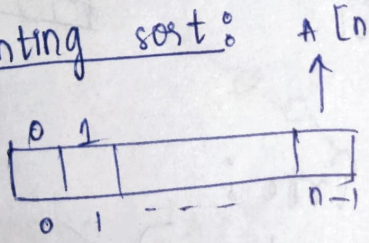
A ← B

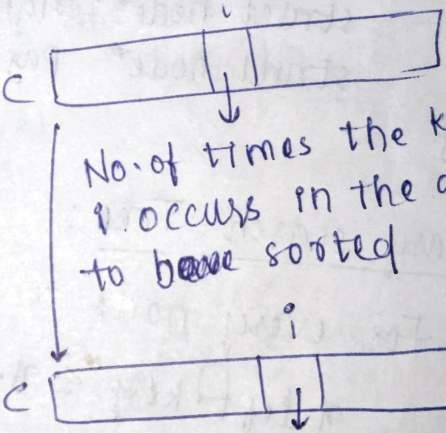
n, no. of elements whose value $\leq n$ is t

* C[i] stores how many yet-to-read elements are there whose key values are $\leq i$.

10/2/26

Counting sort:

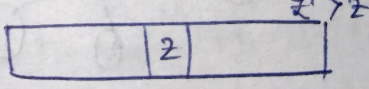





```
for(i=n-1; i>=0; i--){
    B[C[A[i]]-1] = A[i]
    C[A[i]]--;
}
```

C[z']

No. of objects yet to be processed with key value $\leq z'$? X



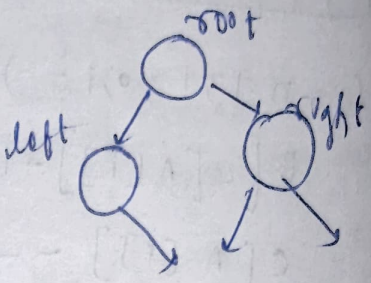
C[i] → No. of objects in the whole list with key value $<i$ + No. of objects yet to be processed whose key value is $=i$.

Dynamic Sets:

INSERT
DELETE
EXTRACT-MIN
MINIMUM
MAXIMUM
SEARCH — $\Omega(h)$

Binary search Tree

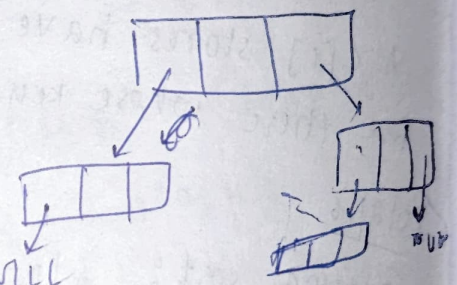
Need not be complete Binary tree.



Binary Tree

At most 2 child nodes

```
struct node {
    int key;
    struct node* left;
    struct node* right;
    struct node* parent;
}
```

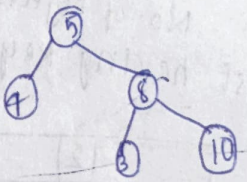


Binary search Tree:

* For every node x of the tree:

$x.left.key \leq x.key \leq x.right.key$

Ex:



for every node y in the left subtree

$y.key \leq x.key$, and

for every node y' in right subtree)

$y.key \geq x.key$.

~~Search(node* x, key)~~

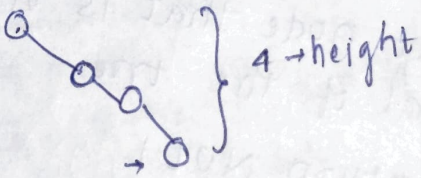
Search(node* x, key2){ → if (x == NULL){ return NULL;

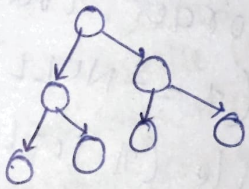
if(x.key == key2){
return x;
}
} Returns a node having key if exists, otherwise returns NULL

else if (x.key > key2){
return (search(left(x), key2));
}
else {
return (search(right(x), key2));
}
}

* Time complexity depends on Height of tree
O(h)

for ex:





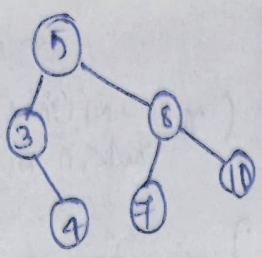
Left - Root → Right

```
IN-ORDER(x){
    if(x==NULL) return;
    INORDER(x.left);
    printf(x.key);
    INORDER(x.right);
}
```

T.C: O(n)
n → No. of nodes

```
PRE-ORDER(x){
    if(x==NULL) return;
    printf(x.key);
    PRE_ORDER(x.left);
    PRE_ORDER(x.right);
}
```

```
POST_ORDER(x){
    if(x==NULL) return;
    POSTORDER(x.left)
    POSTORDER(x.right)
    printf(x.key);
}
```

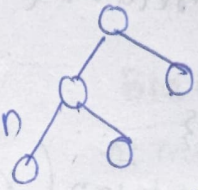


Minimum: Go left and left untill you don't find children on left.

Maximum: Go right and right untill you don't find child on right

23/2/26

Binary Search Trees:



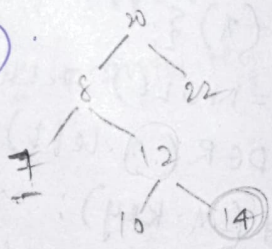
If y is a node in the left subtree of x, y key $\leq$ x key

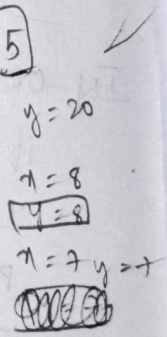
If y is a node in the right subtree of x, y key $\geq$ x key.

MINIMUM(x):

Returns the first node that is visited by an inorder traversal of the tree

```
if (x == NULL) return NULL
while (x.left ≠ NULL)
    x ← x.left
}
return x
```





O(h) → height of tree

SUCCESSOR(x):

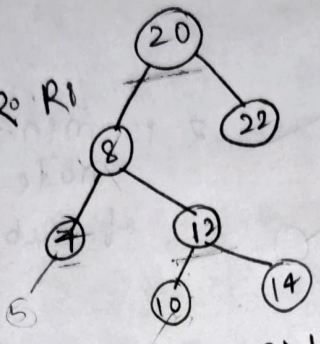
```
if x.right ≠ NULL
    return MINIMUM(x.right)

y = x.p
while (y ≠ NULL AND x = y.right)
    x ← y
    y ← x.p
```

return y
looks for y

INORDER SUCCESSOR → next node in the inorder traversal

for last node → INORDER SUCCESSOR → NULL



L R Ri

for 8 it is 10
10 → 12
14 → 20

for 14 : while loop:
x=14, y=12
x=12, y=8,
x=8, y=20

INORDER TRAVERSAL ⇒ left, key, right

⇒ 7 8 10 12 14 20 22

* INSERT (T, z):
y ← NULL
x ← T.root
while (x != NULL)
y ← x
if (z.key ≤ x.key)
x ← x.left
else
x ← x.right
If (z.key ≤ y.data)
y.left ← z
z.p ← y;
else y.right ← z;
z.p ← y

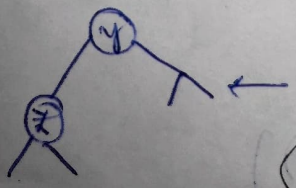
if x = NULL
T.root = z
return

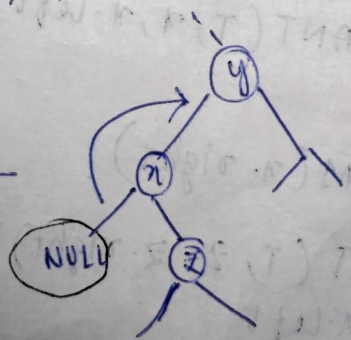
y = Floor of a key
node <= key

O(h)
while loop is always running from root node to child → child → NULL

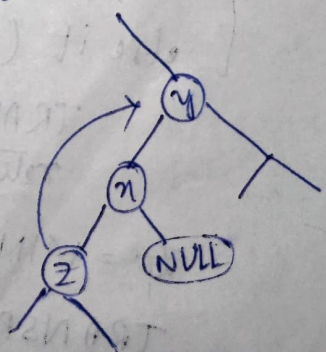
25/2/26

* DELETE (T, x) → pointer to node to be deleted



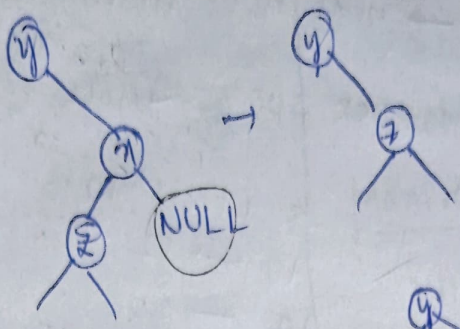


Left sub-tree of y

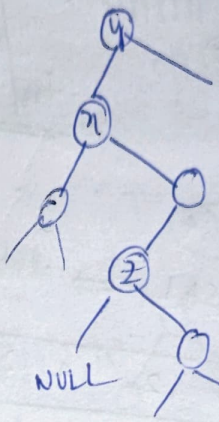


Left subtree of y

Right-subtree



Child's node not NULL:



z is min. node of subtree

```
TRANSPLANT(T, u, v)
y ← u.p
if (y == NULL)
    T.root ← v
else if (u = y.left)
    y.left ← v
else
    y.right ← v
if (v ≠ NULL)
    if (v.p.left = v)
        v.p.left ← NULL
    else
        v.p.right ← NULL
```

(T, u, v)

v could be NULL

u may be root

u is not descendent of v

v ≠ T.root

if (v != NULL)

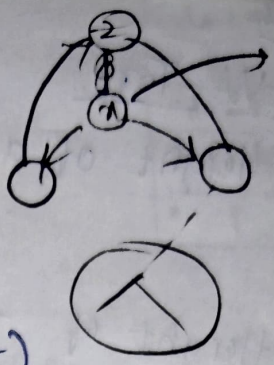
v → parent = u → parent

O(1)

```
DELETE(T, x)
if (x.left = NULL)
    TRANSPLANT(T, x, x.right)
    return
else if (x.right = NULL)
    TRANSPLANT(T, x, x.left)
    return
z = MINIMUM(x.right)
TRANSPLANT(T, z, z.right)
z.left ← x.left
x.left.p ← z
```

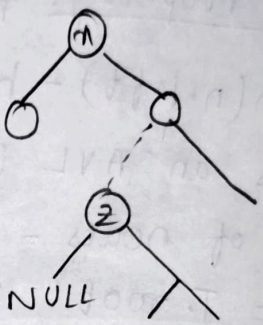
O(h) for minimum

z.right ← x.right
x.right.p ← z
TRANSPLANT(T, x, z)



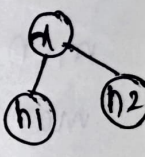
Instead of }

if(z.p ≠ x)
    TRANSPLANT(T, z, z.right)
    z.right ← x.right
z.right.p ← z
z.left ← x.left
x.left.p ← z
TRANSPLANT(T, x, z)



AVL TREES:

Adelson-Velsky, Landis



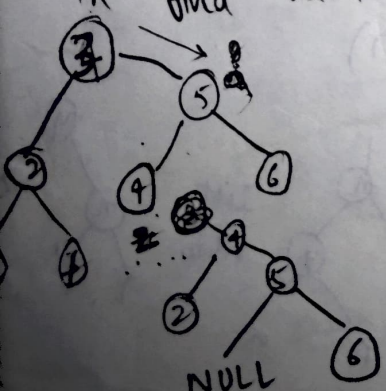
$x.h = \max(x.left.h, x.right.h) + 1$

For every node x,

$|x.left.h - x.right.h| \leq 1$

3-deletion cases:

Left is NULL ⇒ replace x with x.right
Right is NULL ⇒ " y " y.right
Both are not NULL
x find min of x.right = z
if (z → parent != x)



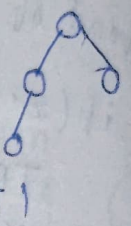
27/2/26

AVL Tree

Height of a node n : Maximum no. of vertices in a path from n to leaf

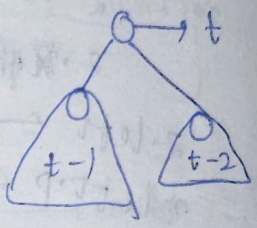
Height of T : Height of root

$h(n) = \max\{h(n.left), h(n.right)\} + 1$



AVL Property: For every node n,

$|h(n.left) - h(n.right)| \leq 1$



T is an AVL tree with height T

No. of nodes = $\Omega(2^t)$

n = T.root, h(n) = t

$\min\{h(n.left), h(n.right)\} \geq t-2$

$\max\{h(n.left), h(n.right)\} = t-1$

f(t) = No. of nodes in an AVL tree of height t

$f(t) \geq f(t-1) + f(t-2)$

$f(0) = 0$

$f(1) = 1$

$f(2) \geq f(0) + f(1) = 1$

$f(3) > f(1) + f(2) = 2$

0, 1, 1, 2, 3, 5, 8, 13, ---

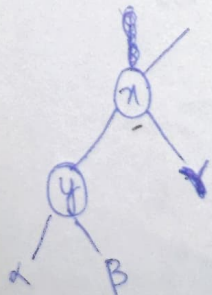
$f(t) = \omega(2^{t/2})$

$f(t) > 2^{t/2}$

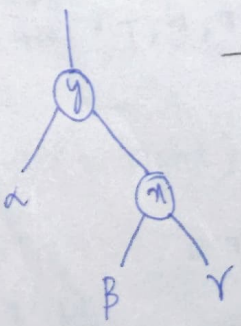
$\log f(t) > t/2$

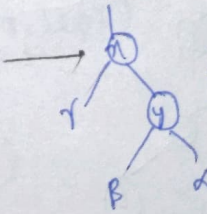
$t < 2\log(f(t))$

Height of an AVL tree of n-nodes $\leq 2\log n = O(\log n)$

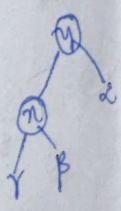


right rotate





left rotate



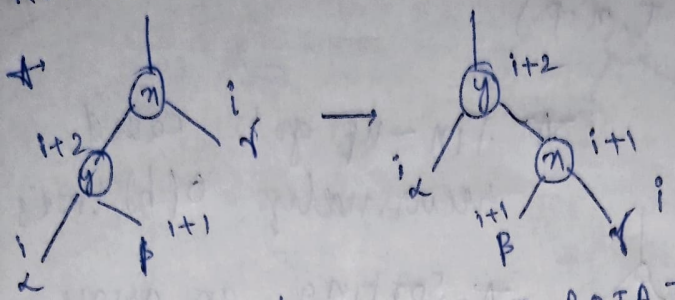
$x \geq values(\beta) \geq y$

x.left.h

x.right.h

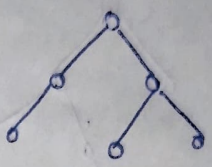
$x.h = \max\{x.left.h, x.right.h\}$

ROTATE_RIGHT(T, x):

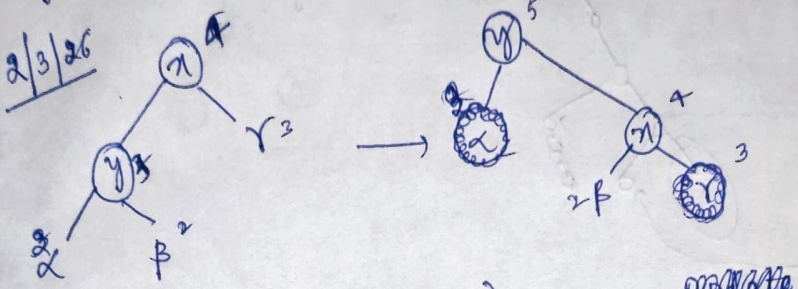


data
h
*left
*right
x	p

y.left.h = y.h − 1 ⇒ RIGHT_ROTATE(T, x)



2/3/26



x.h

height is stored inside the node

ROTATE_RIGHT(T, x)

updateheight(x){

x.h ← max{x.left.h, x.right.h}

}

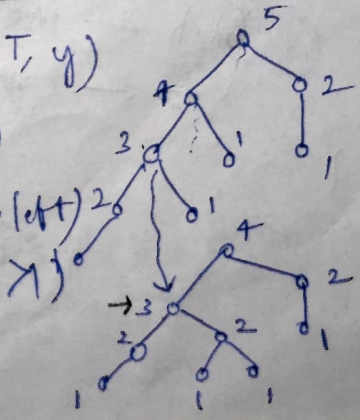
[Also write code for x.left.h or x.right.h should be zero when x.left or x.right is NULL]

height(x) = max{ max{α, β} + 1, γ } + 1

height(y) = max{ max{β, γ} + 1, α } + 1

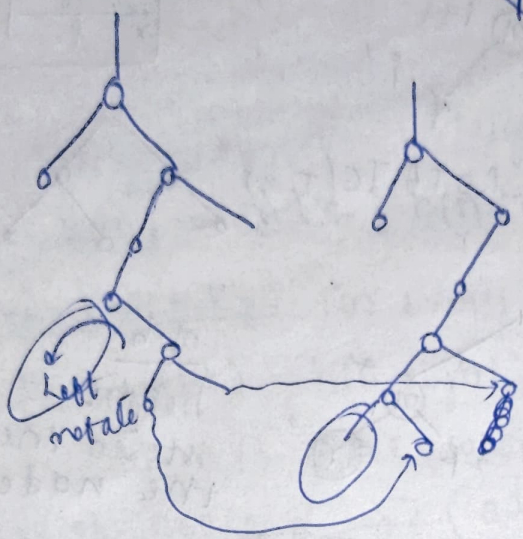
Are they both equal?

FIX_UP(T, x)
if (x == NULL) return
updateheight(x)
if (height(x.left) − height(x.right) > 1)
y ← x.left
left.left ≤ left.right
if (height(y.left) ≠ height(y) − 1)
ROTATE_LEFT(T, y)
ROTATE-RIGHT(T, x)
else if (height(x.right) − height(x.left) > 1)
y ← x.right



else

FIX-UP(T, x.p)

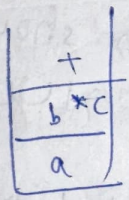


* Fix-up gets called recursively O(h) times

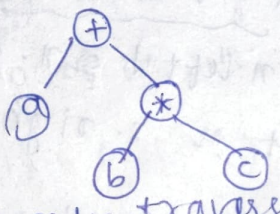
* Sorting an array using B.S.T

Postfix notation

a+b*c



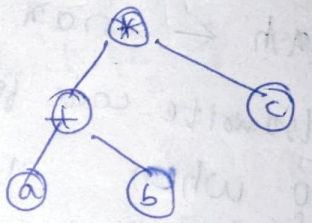
abc*+



Post order traversal:

abc*+

(a+b)*c



ab+c*

ab+c*

ROTATE_RIGHT(T, x) {

y ← x.left

z ← y.right

y ← x.right
z ← y.left
y.left ← x
x.right ← z

y.right ← x

x.left ← z

x.h ← max{height(x.left), height(x.right)}+1

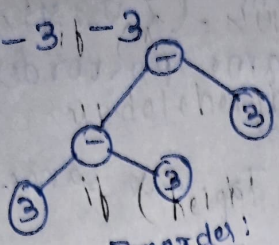
y.h ← max{height(y.left), height(y.right)}+1

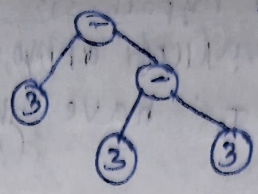
}

6/3

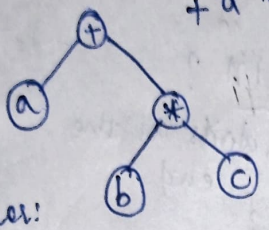
Infix notation

3 - 3 + 3





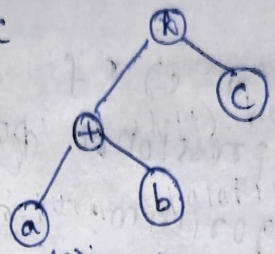
Preorder:
+ a * b c



Inorder:
a + b * c

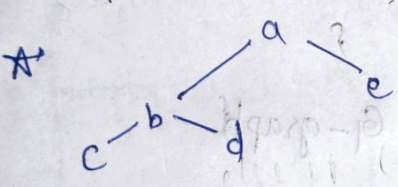
Postorder:
a b c * +

Preorder:
* + a b c

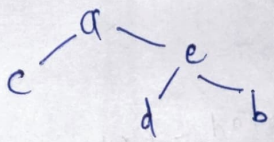


Inorder:
a + b * c

Postorder:
a b + c *



c d b e a



c d b e a

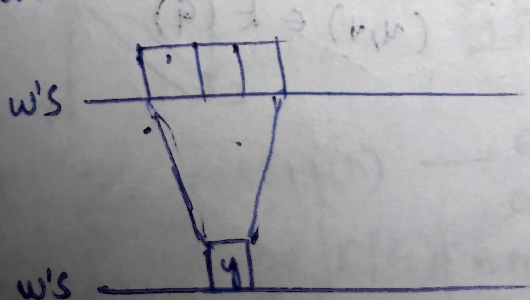
Operands Z

Operator (binary) X

* (Z, X) - tree T
* T is a binary tree having leaves $\subseteq Z$ and non-leaves $\subseteq X$
* Every non-leaf has exactly 2 children

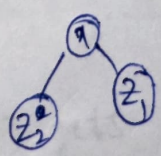
Claim: Two different (Z, X)-trees T_1 & T_2 cannot have the same post-order traversal.

Suppose for the sake of contradiction, the postorder traversal of T_1, T_2 are same



$T_1 - T_2'$

$T_2 - T_2'$



$(Z \cup \{4\}, X)$ - true

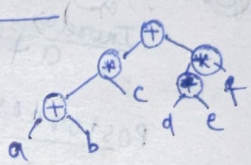
By the inductive hypothesis w' can be the postorder traversal of atmost one $(z \cup \{y\}, x)$-tree. Both T_1' & T_2' have the same postorder traversal w'

T_1' & T_2' are the same $\Rightarrow$ T_1 & T_2 are same

* $(a+b) * c + (d-e) * f$

Convert infix expression to postfix ?

Shunting yard algorithm: operators at the end

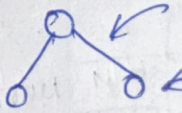


a b + c * d e - f * +

9/3/26

Graphs

G - graph

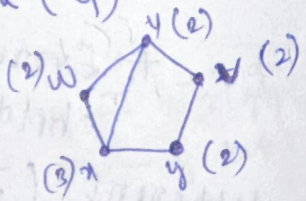


$V(G)$ - vertex set $\quad E(G) \subseteq V(G) \times V(G)$

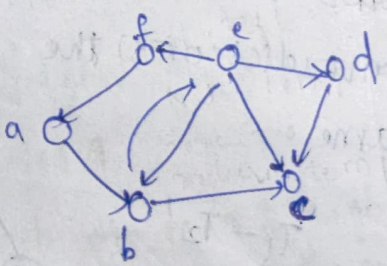
$E(G)$ - edge set

Undirected graphs : $uv \in E(G)$ $\quad \{u, v\} \in E(G)$

degree



Directed graphs :



$E(G) = \{(f, a), (e, b), (b, e), (d, c) \ldots\}$

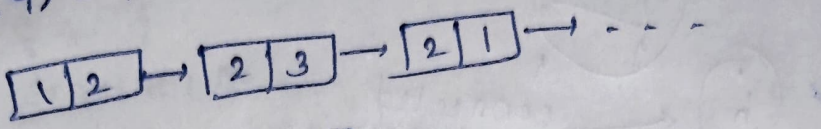
$u, v \in V(G)$

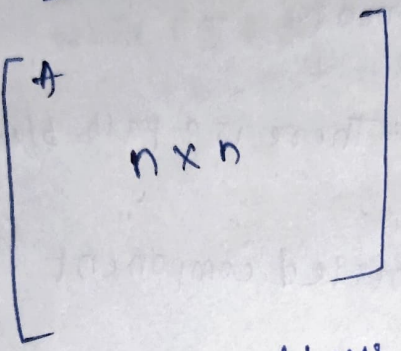
$(u, v) \in E(G)$

Adjacency matrix:

$V(G) = \{1, 2, \ldots, n\}$ $\quad n = |V(G)|$

$E(G) = \{(1,2), \ldots\}$ $\quad m = |E(G)|$



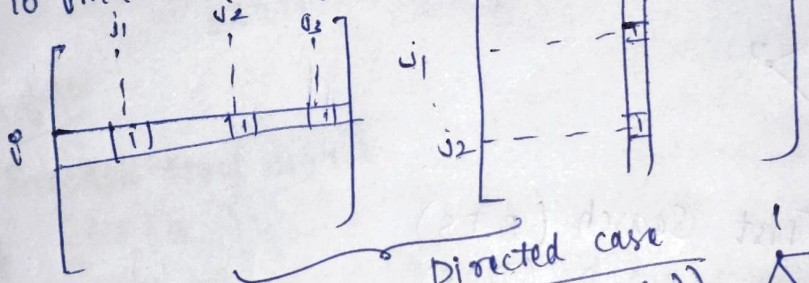


$(-1)(n^2)$

$$Adj = \begin{cases} 1, & \text{if } (n,j) \in E(G) \\ 0, & \text{otherwise} \end{cases}$$

$O(1)$

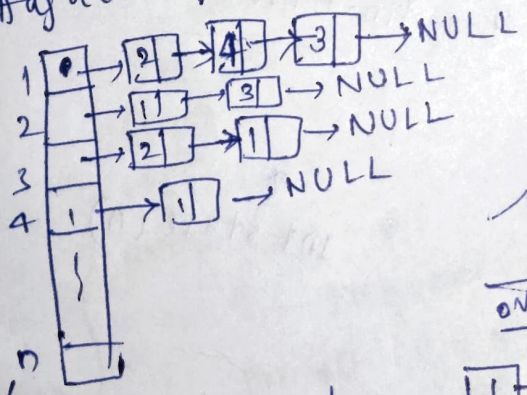
To find the neighbours of i

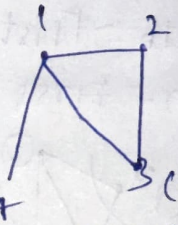


Directed case

$O(deg(n))$

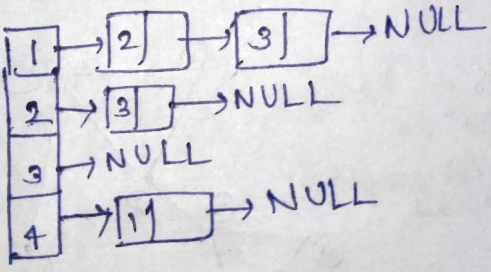
Adjacency list



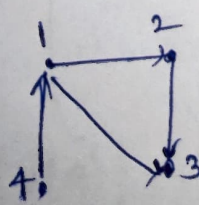


Here every edge gets counted twice
→ one from i & one from j

out neighbours



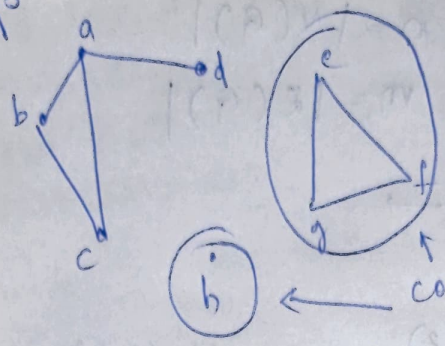
For directed graph



$\sum_{v \in V(G)} outdeg(n) = |E(G)| = m$ → Every edge (i,j) gets counted only once

$\sum_{n \in V(G)} deg(n) = 2|E(G)| = 2m$

11/3



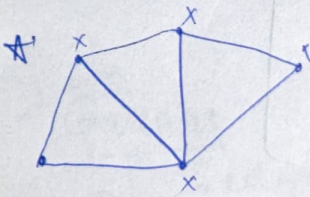
a, b, c, a, d – walk

b, c, a, d – path b/w b & b

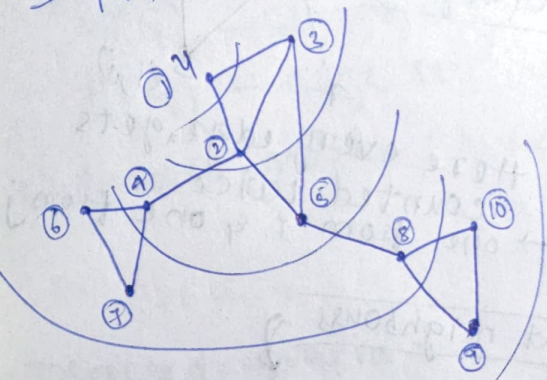
* For some $u \in V(G)$
list all vertices $v \in V(G)$ s.t. There is a path b/w u & v

(OR)

list the vertices in the connected component containing u.



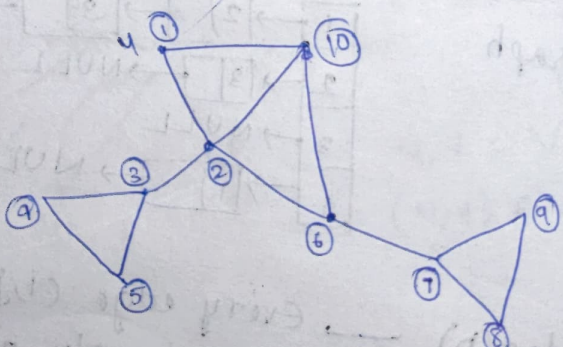
Breadth - First Search (BFS)
Depth - First Search (DFS)



Breadth-First search

int state (n)

DFS:



```
BFS(G, s):
    for all u ∈ V(G):
        state(u) ← 0
    Q ← ϕ
    Enqueue(Q, s)
    while (Q ≠ ϕ)
        u ← Dequeue(Q)
        for all v ∈ Adj(u)
            if (state(u) = 0)
                state(v) ← 1
                Enqueue(Q, v)
    state(u) ← 2
```

13/3/24

Breadth First Search

```
BFS(G,s):
    for all u = V(G):
        state(u) ← 0 ; d[u] ← ∞;
    Q ← ϕ
    Enqueue(Q, s)
    state[s] ← 1
    while (Q != ϕ)
        u ← Dequeue(Q)
        for all v ∈ Adj[u]
            if (state(v) = 0)
                ENQUEUE(Q, v);
                state[v] ← 1
                d[v] ← d[u] + 1
        state(u) ← 2
```

O(m)

vertex v has visited (you can point here

$|V(G)| \leftarrow n$

$|E(G)| \leftarrow m$

$O(n+m)$

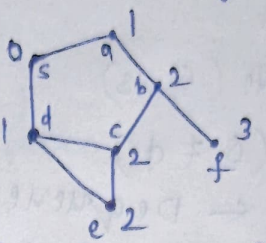
Linear time

Distance of u from s

→ length of a shortest path from s to u containing min. no. of edges

No. of edges (m)

→ $\delta(s,u)$

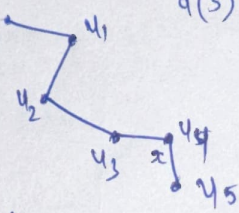


→ After BFS(G,s) terminates, ~~$\forall u \in V(G)$~~,

$d[u] = \delta(s,u)$.

What if $d[u] < \delta(s,u)$? / $d[u] >= \delta(s,u)$

eg:



$d(s) = 0$

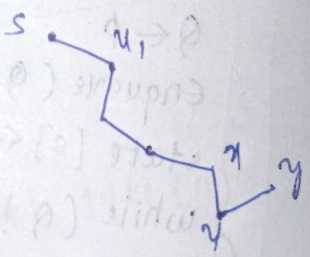
$\delta(s,u_i) = ?$

If $\delta(s,u) = 0$ then $s = u$

Let u be the vertex having $d[u] > \delta(s,u)$ s.t $d(u)$ is as small as possible

$\delta(s,x) = \delta(s,u) - 1$ $\quad \delta(s,x) \leq d[x]$

$d \rightarrow$ something calculated by the BFS algorithm.



$\Rightarrow d(u) = d[y] + 1$

$d[y] >= \delta(s,y)$

$d[u] \geq \delta(s,y) + 1$

$d[x] \geq \delta(s,x)$

$\boxed{\delta(s,u) > \delta(s,y) + 1}$ How?